

**,VISVESVARAYA TECHNOLOGICAL
UNIVERSITY**

“JnanaSangama”, Belgaum -590014, Karnataka.



**LAB REPORT
on**

Machine Learning (23CS6PCMAL)

Submitted by

Rayhan Sadat (1BM23CS264)

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **Rayhan Sadat (1BM23CS264)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Rashmi H Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	---

Index

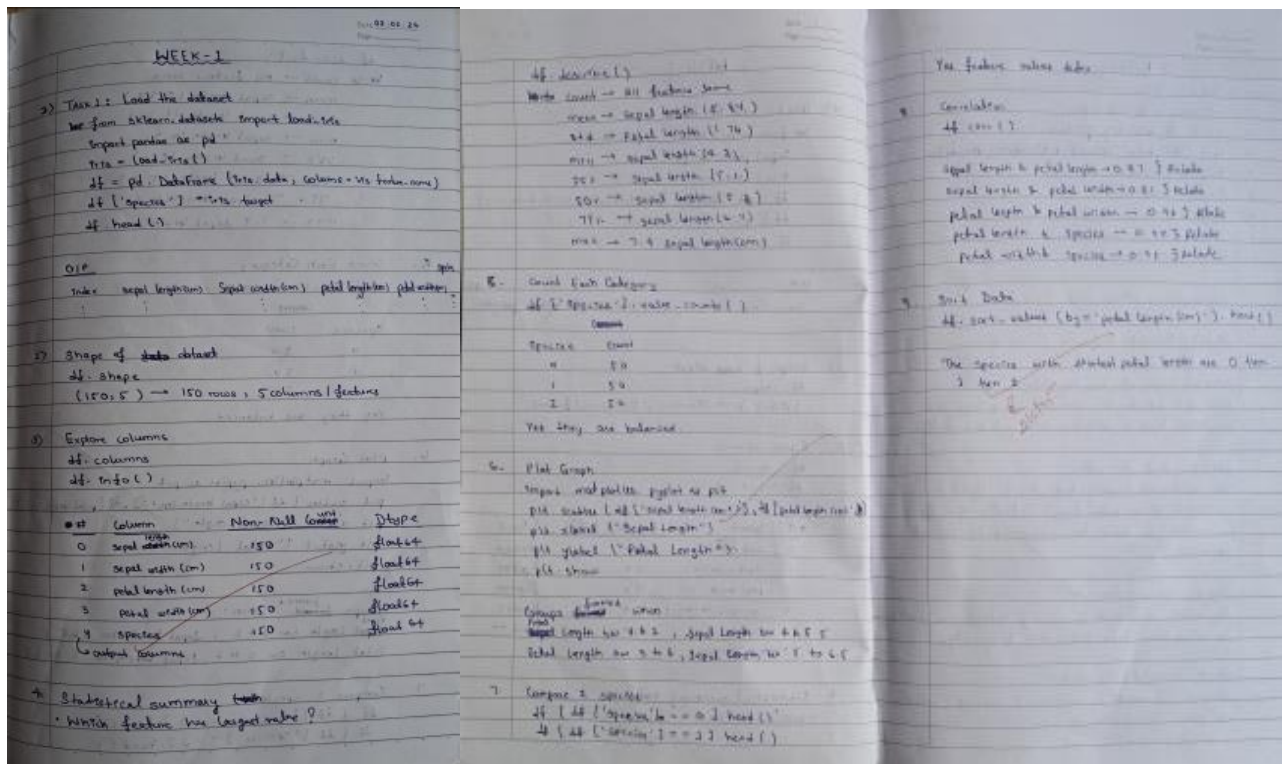
Sl. No.	Date	Experiment Title	Page No.
1	21-2-2025	Write a python program to import and export data using Pandas library functions	4
2	3-3-2025	Demonstrate various data pre-processing techniques for a given dataset	6
3	10-3-2025	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	7
4	17-3-2025	Build Logistic Regression Model for a given dataset	7
5	24-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	9
6	7-4-2025	Build KNN Classification model for a given dataset.	11
7	21-4-2025	Build Support vector machine model for a given dataset	17
8	5-5-2025	Implement Random forest ensemble method on a given dataset.	21
9	5-5-2025	Implement Boosting ensemble method on a given dataset.	24
10	12-5-2025	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	29
11	12-5-2025	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	32

Github Link:

https://github.com/RayhanSadat26/ML_Lab

Program 1

Screenshot



Code:

<

In [11]:	<pre>df['species'].value_counts()</pre>								
Out[11]:	<table> <tr> <th>species</th><th>count</th></tr> <tr><td>0</td><td>50</td></tr> <tr><td>1</td><td>50</td></tr> <tr><td>2</td><td>50</td></tr> </table> dtype: int64	species	count	0	50	1	50	2	50
species	count								
0	50								
1	50								
2	50								
In [13]:	<pre>plt.scatter(df['sepal length (cm)'], df['petal length (cm)']) plt.xlabel('sepal length (cm)') plt.ylabel('petal length (cm)') plt.show()</pre>								

```
In [15]: plt.scatter(df['sepal width (cm)',df['petal width (cm)'])
plt.xlabel('sepal width(cm)')
plt.ylabel('petal width(cm)')
plt.show()
```

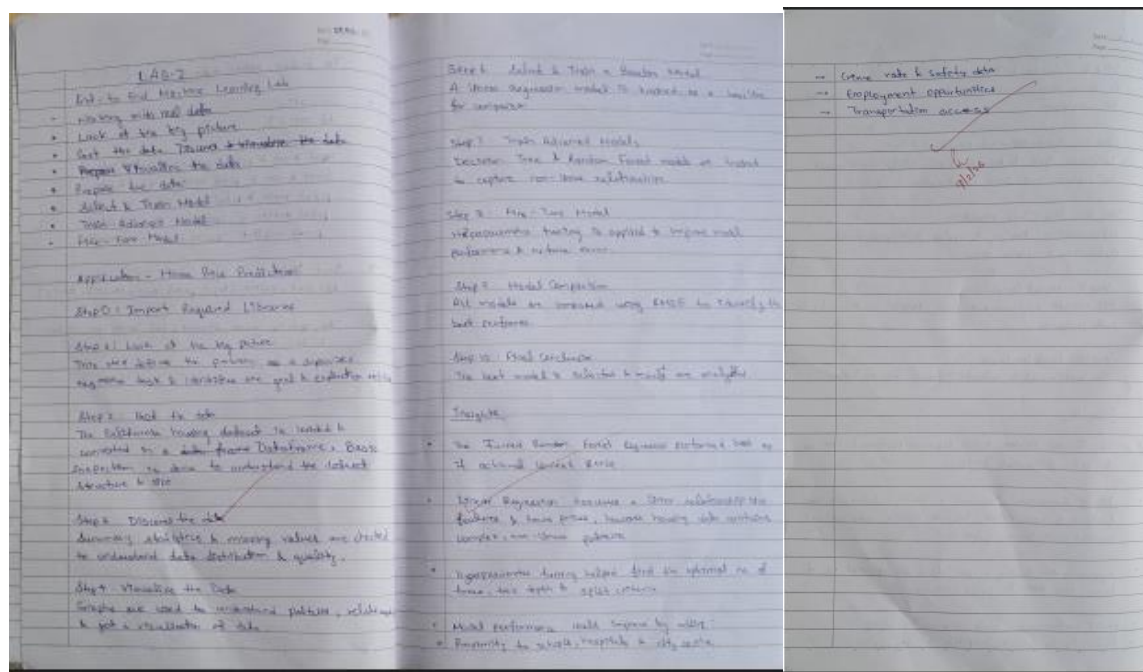
```
In [16]: df[df.species==0].head()
```

```
Out[16]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

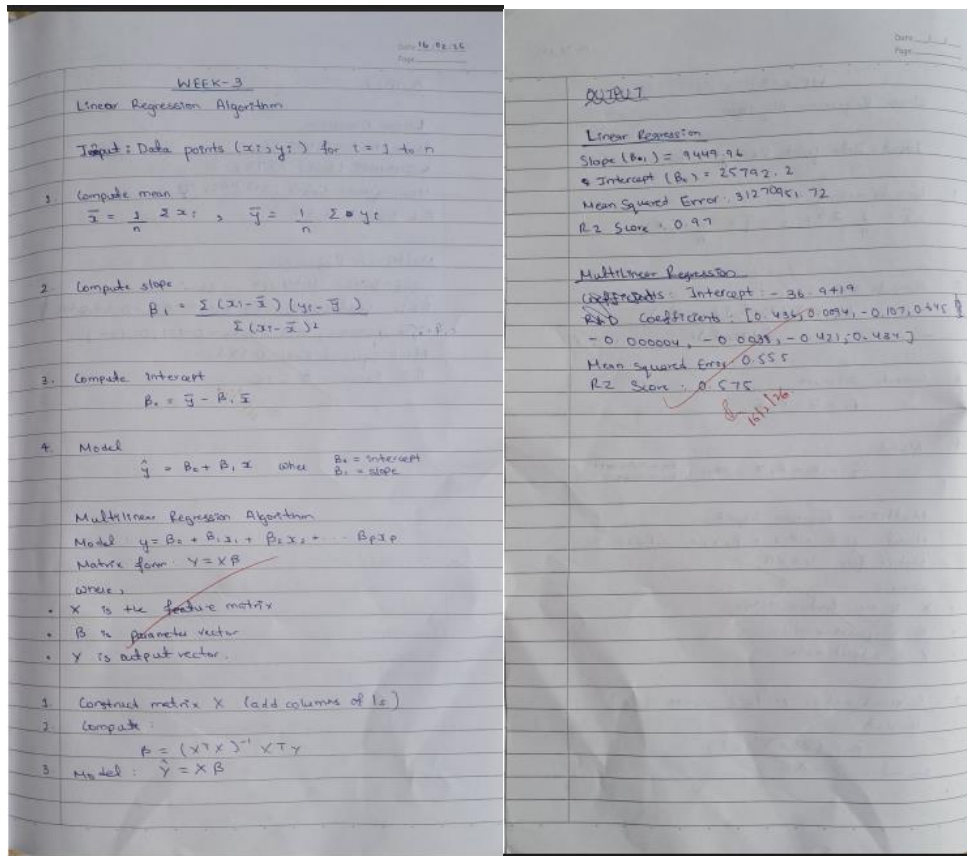
Program 2

Screenshots:



Program 3 & 4

Screenshot



Code:

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

Load dataset

```
data = fetch_california_housing()
X = data.data[:, [0]] # Use only MedInc (first feature)
y = data.target
```

Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Train

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])
```

```
print("R^2:", model.score(X_test, y_test))
```

O/P:

Intercept: 0.4445972916907872

Slope: 0.4193384939381273

R^2: 0.45885918903846656

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
# Load dataset
```

```
data = fetch_california_housing()
```

```
X = data.data # All features
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Train
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
print("Intercept:", model.intercept_)
```

```
print("Coefficients:", model.coef_)
```

```
print("R^2:", model.score(X_test, y_test))
```

O/P:

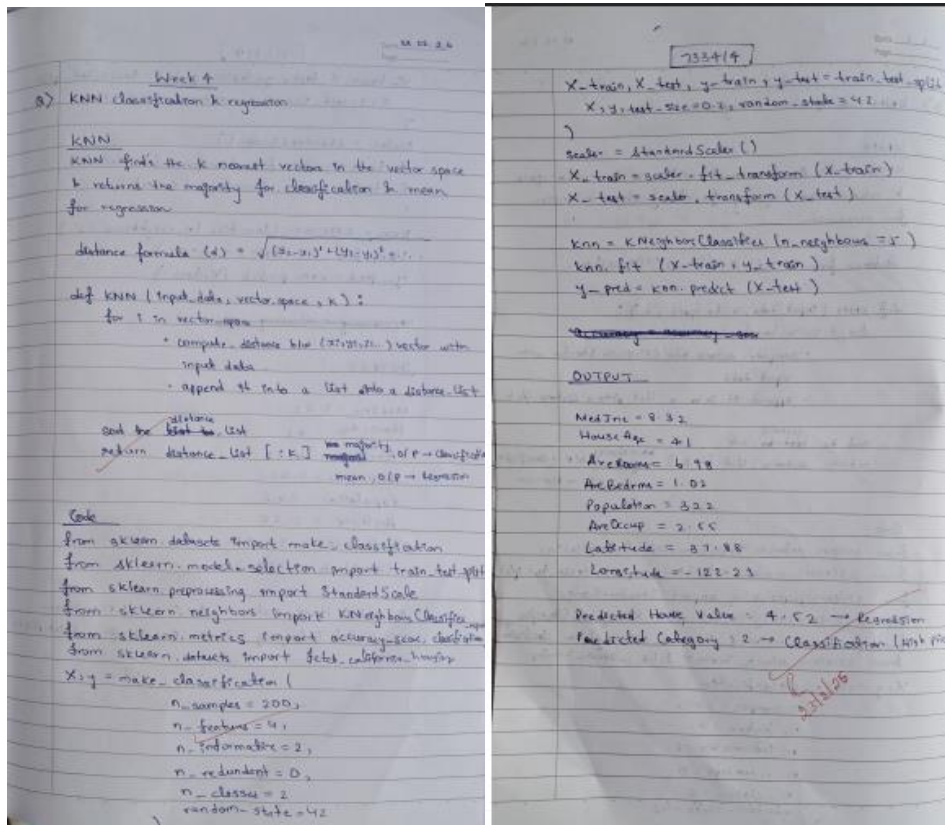
Intercept: -37.023277706063794

Coefficients: [4.48674910e-01 9.72425752e-03 -1.23323343e-01 7.83144907e-01
-2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]

R^2: 0.5757877060324524

Program 5

Screenshot



Code:

import numpy as np

from sklearn.neighbors **import** KNeighborsClassifier, KNeighborsRegressor

X = np.array([[100], [105], [98], [150], [160], [155]])

y_class = np.array([0, 0, 0, 1, 1, 1])

y_reg = np.array([100, 105, 98, 150, 160, 155])

1) KNN Classification

knn_classifier = KNeighborsClassifier(n_neighbors=3)

knn_classifier.fit(X, y_class)

new_ball = np.array([[110]])

```
class_prediction = knn_classifier.predict(new_ball)

if class_prediction[0] == 0:

    print("Classification: Red Ball")

else:

    print("Classification: Blue Ball")

# 2) KNN Regression

knn_regressor = KNeighborsRegressor(n_neighbors=3)

knn_regressor.fit(X, y_reg)

weight_prediction = knn_regressor.predict(new_ball)

print("Regression (Average Predicted Weight):", weight_prediction[0])
```

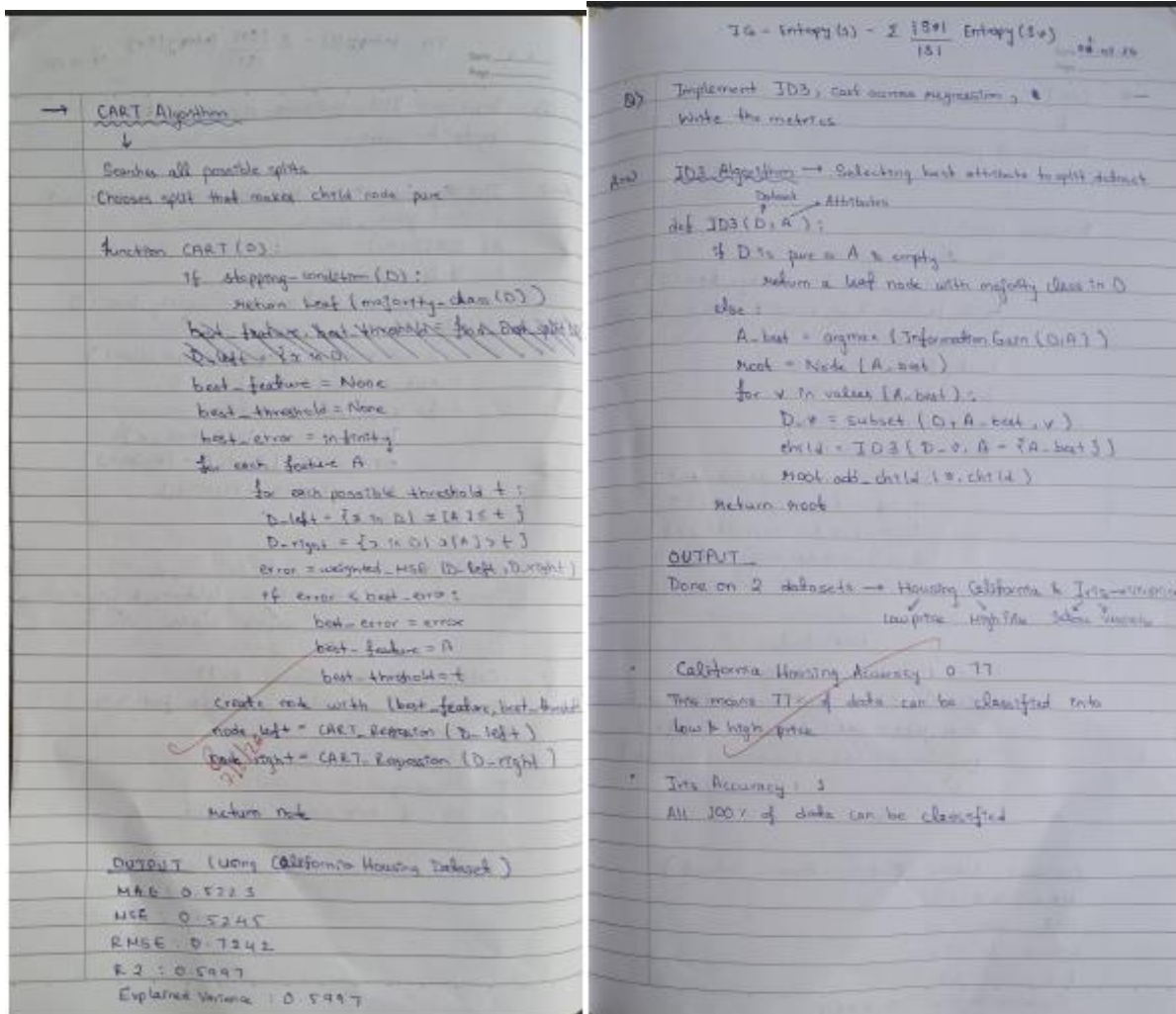
O/P:

Classification: Red Ball

Regression (Average Predicted Weight): 101.0

Program 6

Screenshots:



Code:

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.datasets import load_iris, load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

```

def evaluate_classification(X, y, dataset_name):

    print("\n=====")

    print("Dataset:", dataset_name)

    print("=====")

    # Split dataset

    X_train, X_test, y_train, y_test = train_test_split(

        X, y, test_size=0.3, random_state=42

    )

    # ID3 → entropy criterion

    model = DecisionTreeClassifier(criterion="entropy", random_state=42)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    # Error Metrics

    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(y_test, y_pred, average='weighted')

    recall = recall_score(y_test, y_pred, average='weighted')

    f1 = f1_score(y_test, y_pred, average='weighted')

    cm = confusion_matrix(y_test, y_pred)

    print("Accuracy:", accuracy)

    print("Precision:", precision)

    print("Recall:", recall)

    print("F1 Score:", f1)

    print("\nConfusion Matrix:")

    print(cm)

```

```
# Dataset 1: Iris
```

```
iris = load_iris()
```

```
evaluate_classification(iris.data, iris.target, "Iris Dataset")
```

```
# Dataset 2: Breast Cancer
```

```
cancer = load_breast_cancer()
```

```
evaluate_classification(cancer.data, cancer.target, "Breast Cancer Dataset")
```

O/P:

```
=====
```

Dataset: Iris Dataset

```
=====
```

Accuracy: 0.9777777777777777

Precision: 0.9793650793650793

Recall: 0.9777777777777777

F1 Score: 0.9777448559670783

Confusion Matrix:

```
[[19  0  0]
```

```
 [ 0 13  0]
```

```
 [ 0  1 12]]
```

```
=====
```

Dataset: Breast Cancer Dataset

```
=====
```

Accuracy: 0.9649122807017544

Precision: 0.9649541140481607

Recall: 0.9649122807017544

F1 Score: 0.9647902680643604

Confusion Matrix:

```
[[ 59  4]
```

```
 [ 2 106]]
```

Decision Tree Regression using CART Algorithm

```
import numpy as np
```

```
from sklearn.datasets import fetch_california_housing, load_diabetes
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
def evaluate_regression(X, y, dataset_name):
```

```
    print("\n=====")
```

```
    print("Dataset:", dataset_name)
```

```
    print("=====")
```

```
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.3, random_state=42
```

```
    )
```

CART $\rightarrow$ squared_error criterion

```
    model = DecisionTreeRegressor(
```

```
        criterion="squared_error",
```

```

    random_state=42

)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# Error Metrics

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("MAE:", mae)

print("MSE:", mse)

print("RMSE:", rmse)

print("R2 Score:", r2)

# Dataset 1: California Housing

housing = fetch_california_housing()

evaluate_regression(housing.data, housing.target, "California Housing Dataset")

# Dataset 2: Diabetes

diabetes = load_diabetes()

evaluate_regression(diabetes.data, diabetes.target, "Diabetes Dataset")

O/P:

=====

Dataset: California Housing Dataset

=====

MAE: 0.46904822189922485

```

MSE: 0.5280096503174904

RMSE: 0.7266427253592307

R2 Score: 0.5977192261218356

=====

Dataset: Diabetes Dataset

=====

MAE: 60.015037593984964

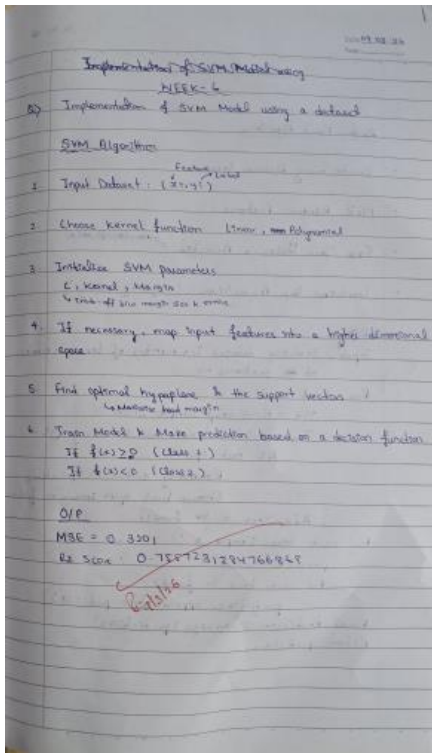
MSE: 5697.789473684211

RMSE: 75.48370336492647

R2 Score: -0.05547696148647652

Program 7

Screenshots:



Code:

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Load dataset
```

```
iris = load_iris()
```

```

X = iris.data

y = iris.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Feature scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# Create SVM model

svm_model = SVC(kernel='linear')

# Train model

svm_model.fit(X_train, y_train)

# Predictions

y_pred = svm_model.predict(X_test)

# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")

print(classification_report(y_test, y_pred))

O/P:

Accuracy: 0.9666666666666667

Confusion Matrix

```

```
[[10 0 0]
```

```
[ 0 8 1]
```

```
[ 0 0 11]]
```

Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy		0.97		30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn import datasets
```

```
from sklearn.svm import SVC
```

```
iris = datasets.load_iris()
```

```
X = iris.data[:, 2:4] # petal length and petal width
```

```
y = iris.target
```

```
model = SVC(kernel='linear')
```

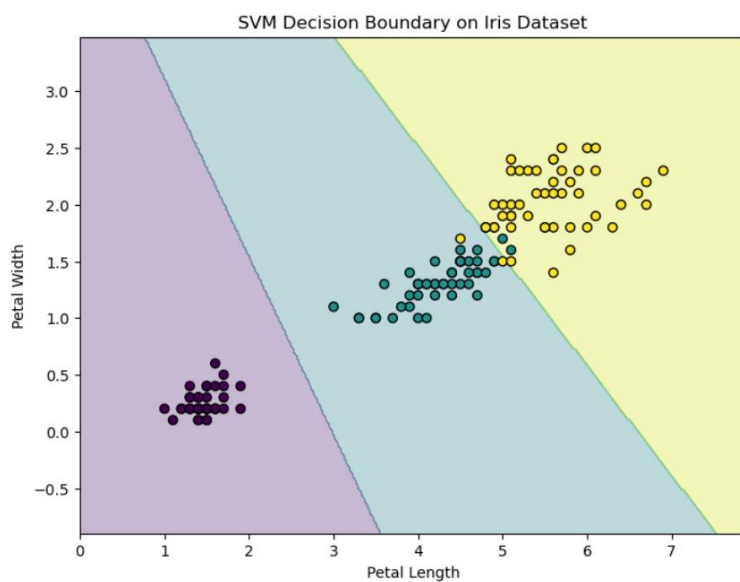
```
model.fit(X, y)
```

```
SVC(kernel='linear')
```

```

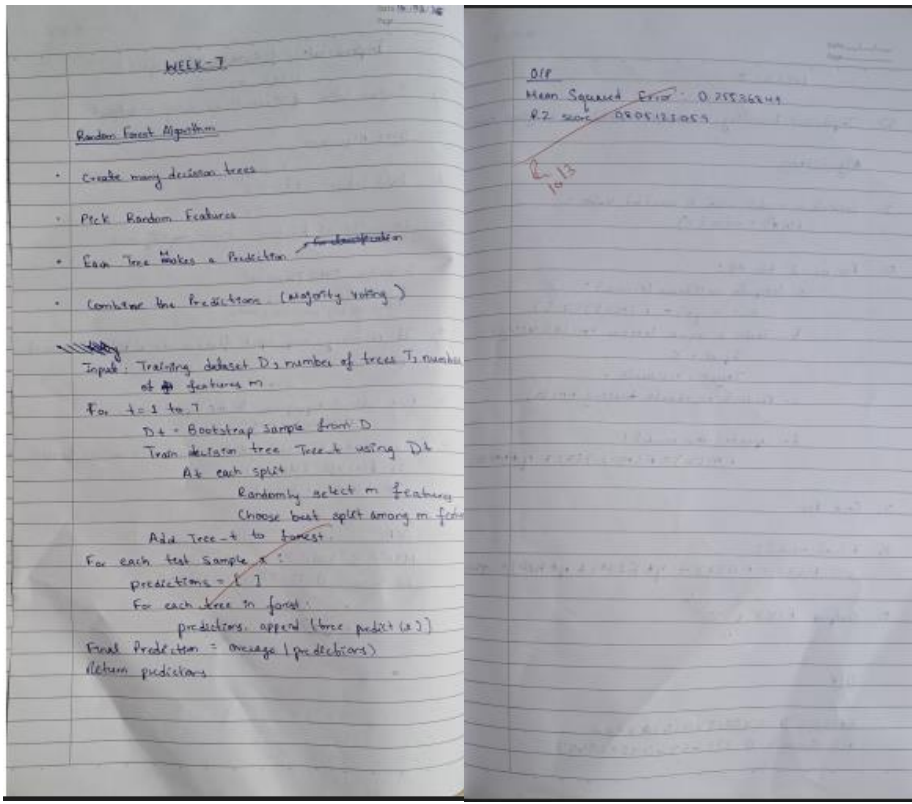
x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1
y_min, y_max = X[:,1].min() - 1, X[:,1].max() + 1
xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.02),
    np.arange(y_min, y_max, 0.02)
)
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.figure(figsize=(8,6))
plt.contourf(xx, yy, Z, alpha=0.3)
scatter = plt.scatter(X[:,0], X[:,1], c=y, edgecolors='k')
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("SVM Decision Boundary on Iris Dataset")
plt.show()

```



Program 8

Screenshots



Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report

import warnings

warnings.filterwarnings('ignore')

# Load dataset

titanic_data = pd.read_csv('titanic.csv')

# Remove rows where Survived is missing
```

```

titanic_data = titanic_data.dropna(subset=['Survived'])

# Select features and target

X = titanic_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']].copy()
y = titanic_data['Survived']

# Encode categorical variable

X['Sex'] = X['Sex'].map({'female': 0, 'male': 1})

# Fill missing Age values with median

X['Age'] = X['Age'].fillna(X['Age'].median())

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Random Forest model

rf_classifier = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train model

rf_classifier.fit(X_train, y_train)

# Make predictions

y_pred = rf_classifier.predict(X_test)

# Evaluate model

accuracy = accuracy_score(y_test, y_pred)

classification_rep = classification_report(y_test, y_pred)

```

```

print(f'Accuracy: {accuracy:.2f}')

print("\nClassification Report:\n", classification_rep)

# Test with a sample passenger

sample = X_test.iloc[[0]]

prediction = rf_classifier.predict(sample)

sample_dict = sample.iloc[0].to_dict()

print("\nSample Passenger:", sample_dict)

print("Predicted Survival:", "Survived" if prediction[0] == 1 else "Did Not Survive")

Accuracy: 0.80

```

Classification Report:

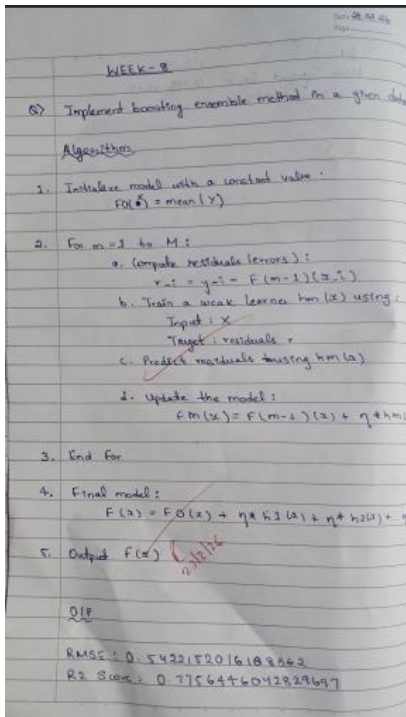
	precision	recall	f1-score	support
0	0.82	0.85	0.83	105
1	0.77	0.73	0.75	74
accuracy			0.80	179
macro avg	0.79	0.79	0.79	179
weighted avg	0.80	0.80	0.80	179

Sample Passenger: {'Pclass': 3.0, 'Sex': 1.0, 'Age': 28.0, 'SibSp': 1.0, 'Parch': 1.0, 'Fare': 15.2458}

Predicted Survival: Did Not Survive

Program 9

Screenshots



Code:

Step 1: Import libraries

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import AdaBoostClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```

# Step 2: Load dataset

data = load_breast_cancer()

X = data.data

y = data.target

# Step 3: Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# Step 4: Create base learner

base_model = DecisionTreeClassifier(max_depth=1)

# Step 5: Create AdaBoost model

model = AdaBoostClassifier(

    estimator=base_model,

    n_estimators=50,

    learning_rate=1.0,

    random_state=42

)

# Step 6: Train model

model.fit(X_train, y_train)

# Step 7: Predictions

y_pred = model.predict(X_test)

# Step 8: Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

```

print("\nClassification Report:\n", classification_report(y_test, y_pred))

# -----

#  VISUALIZATION 1: Confusion Matrix Heatmap

# -----

cm = confusion_matrix(y_test, y_pred)

plt.figure()

plt.imshow(cm)

plt.title("Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

for i in range(len(cm)):

    for j in range(len(cm)):

        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.colorbar()

plt.show()

# -----

#  VISUALIZATION 2: Error Reduction Plot

# -----

errors = []

for y_pred_stage in model.staged_predict(X_test):

    errors.append(np.mean(y_pred_stage != y_test))

plt.figure()

plt.plot(errors)

plt.title("Error Reduction over Boosting Iterations")

```

```

plt.xlabel("Number of Estimators")

plt.ylabel("Error")

plt.show()

# -----

#  VISUALIZATION 3: PCA Decision Boundary

# -----

# Reduce to 2D

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)

# Train again on PCA data

model_pca = AdaBoostClassifier(

    estimator=DecisionTreeClassifier(max_depth=1),

    n_estimators=50,

    random_state=42

)

model_pca.fit(X_pca, y)

# Plot boundary

x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1

y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1

xx, yy = np.meshgrid(

    np.linspace(x_min, x_max, 200),

    np.linspace(y_min, y_max, 200)

)

Z = model_pca.predict(np.c_[xx.ravel(), yy.ravel()])

```

```

Z = Z.reshape(xx.shape)

plt.figure()

plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)

plt.title("AdaBoost Decision Boundary (PCA Reduced Data)")

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.show()

```

O/P:

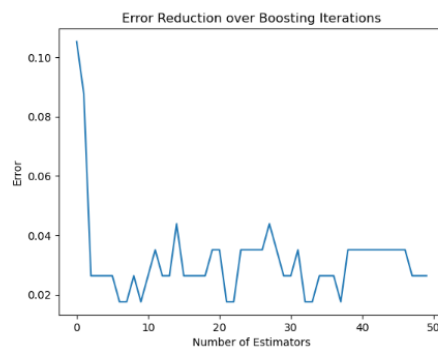
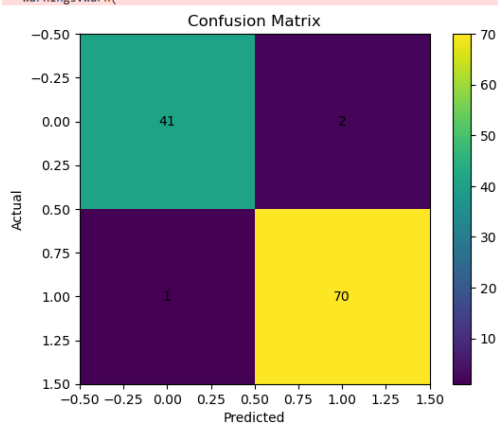
Accuracy: 0.9736842105263158

Confusion Matrix:
[[41 2]
[1 70]]

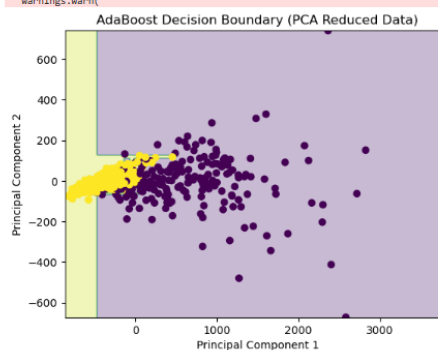
Classification Report:

	precision	recall	f1-score	support
0	0.98	0.95	0.96	43
1	0.97	0.99	0.98	71
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\ensemble_weight_boosting.py:527: FutureWarning:
warnings.warn(

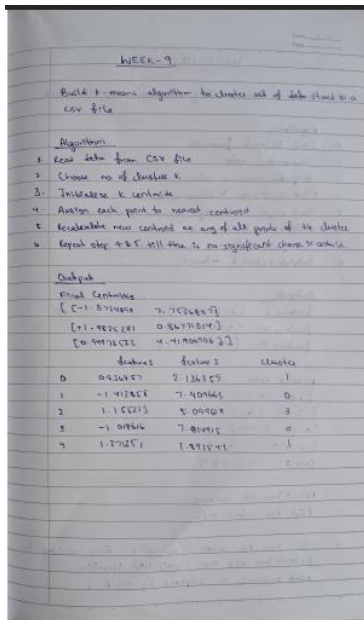


C:\ProgramData\anaconda3\Lib\site-packages\sklearn\ensemble_weight_boosting.py:527: FutureWarning:
warnings.warn(



Program 10

Screenshots



Code:

```
import pandas as pd

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer

df = X.copy()

# Drop ID

df = df.drop(columns=['status_id'], errors='ignore')

# Convert datetime → extract features

df['status_published'] = pd.to_datetime(df['status_published'])

df['hour'] = df['status_published'].dt.hour

df['day'] = df['status_published'].dt.dayofweek

df = df.drop(columns=['status_published'])
```

```

# Separate categorical & numerical

categorical_cols = ['status_type']

numerical_cols = [col for col in df.columns if col not in categorical_cols]

# ColumnTransformer with OneHotEncoder

preprocessor = ColumnTransformer([

    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),

    ('num', StandardScaler(), numerical_cols)

])

# Transform data

scaled_data = preprocessor.fit_transform(df)

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

inertia = []

K_range = range(1, 11)

for k in K_range:

    kmeans = KMeans(n_clusters=k, random_state=42)

    kmeans.fit(scaled_data)

    inertia.append(kmeans.inertia_)

plt.plot(K_range, inertia, marker='o')

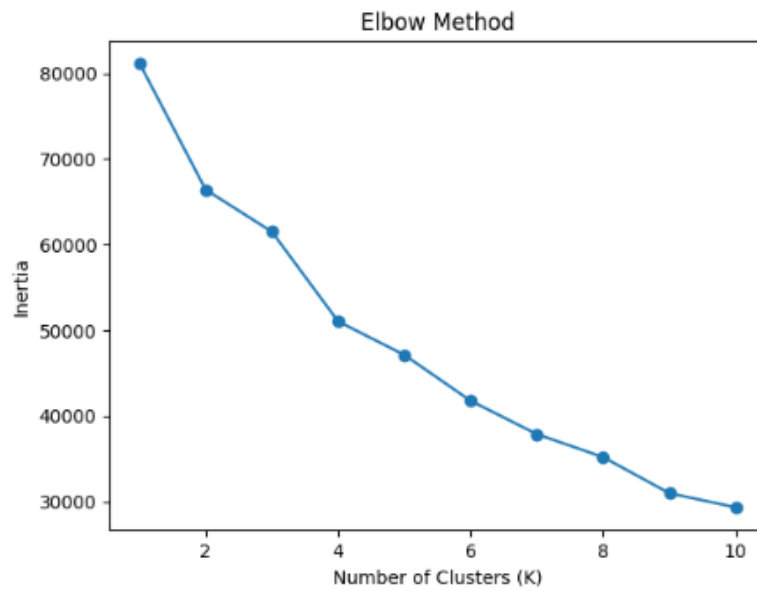
plt.xlabel('Number of Clusters (K)')

plt.ylabel('Inertia')

plt.title('Elbow Method')

plt.show()

```



```
In [27]: from sklearn.cluster import KMeans
          from sklearn.decomposition import PCA

          # Choose K from elbow
          kmeans = KMeans(n_clusters=4, random_state=42)
          clusters = kmeans.fit_predict(scaled_data)

          # PCA
          pca = PCA(n_components=3)
          pca_data = pca.fit_transform(scaled_data)

          import matplotlib.pyplot as plt

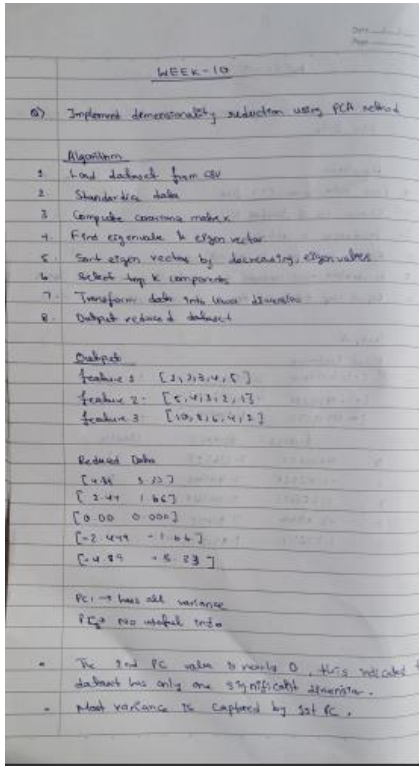
          plt.scatter(pca_data[:, 0], pca_data[:, 1], c=clusters)
          plt.xlabel('PCA 1')
          plt.ylabel('PCA 2')
          plt.title('K-Means Clusters')
          plt.show()

          # Add cluster labels back
          df['Cluster'] = clusters
          print(df.groupby('Cluster').mean(numeric_only=True))
```



Program 11

Screenshots



Code:

```
import numpy as np

import time

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.linear_model import LinearRegression
```

```

from sklearn.pipeline import Pipeline

from sklearn.metrics import mean_squared_error

# Load data

data = fetch_california_housing()

X, y = data.data, data.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# ----- WITHOUT PCA -----

start = time.time()

pipe_no_pca = Pipeline([

    ('scaler', StandardScaler()),

    ('model', LinearRegression())

])

pipe_no_pca.fit(X_train, y_train)

y_pred = pipe_no_pca.predict(X_test)

mse_before = mean_squared_error(y_test, y_pred)

time_no_pca = time.time() - start

# ----- WITH PCA (95%) -----

start = time.time()

pipe_pca = Pipeline([

    ('scaler', StandardScaler()),

    ('pca', PCA(n_components=0.95)),

```

```

    ('model', LinearRegression())

])

pipe_pca.fit(X_train, y_train)

y_pred_pca = pipe_pca.predict(X_test)

mse_after = mean_squared_error(y_test, y_pred_pca)

time_pca = time.time() - start

# Extract PCA info

pca_step = pipe_pca.named_steps['pca']

print("==== RESULTS ====")

print(f'Original features: {X.shape[1]}')

print(f'PCA components (95% variance): {pca_step.n_components_}')

print(f'Explained variance: {pca_step.explained_variance_ratio_.sum():.4f}')

print("\n--- Model Performance ---")

print(f'MSE before PCA: {mse_before:.4f}')

print(f'MSE after PCA: {mse_after:.4f}')

print("\n--- Training Time ---")

print(f'Time without PCA: {time_no_pca:.4f} sec')

print(f'Time with PCA: {time_pca:.4f} sec')

# ----- MULTIPLE PCA SETTINGS -----

components = [2, 4, 6, 8]

mse_list = []

for n in components:

    pipe = Pipeline([

        ('scaler', StandardScaler()),

```

```

    ('pca', PCA(n_components=n)),

    ('model', LinearRegression())

])

pipe.fit(X_train, y_train)

y_pred_n = pipe.predict(X_test)

mse_list.append(mean_squared_error(y_test, y_pred_n))

print("\n--- PCA Component vs MSE ---")

for c, m in zip(components, mse_list):

    print(f'Components: {c}, MSE: {m:.4f}')

```

O/P:

Original features: 8

PCA components (95% variance): 6

Explained variance: 0.9837

--- Model Performance ---

MSE before PCA: 0.5559

MSE after PCA: 0.6714

--- Training Time ---

Time without PCA: 0.0501 sec

Time with PCA: 0.0527 sec

--- PCA Component vs MSE ---

Components: 2, MSE: 1.2946

Components: 4, MSE: 0.7409

Components: 6, MSE: 0.6714

Components: 8, MSE: 0.5559

```

import matplotlib.pyplot as plt

import numpy as np

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

# Scale full data for visualization

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Fit PCA

pca_full = PCA()

pca_full.fit(X_scaled)

explained = pca_full.explained_variance_ratio_

cumulative = np.cumsum(explained)

# ---- Plot 1: Individual variance ----

plt.figure()

plt.plot(range(1, len(explained)+1), explained, marker='o')

plt.xlabel("Principal Component")

plt.ylabel("Explained Variance Ratio")

plt.title("Variance Captured by Each Component")

plt.grid()

plt.show()

# ---- Plot 2: Cumulative variance ----

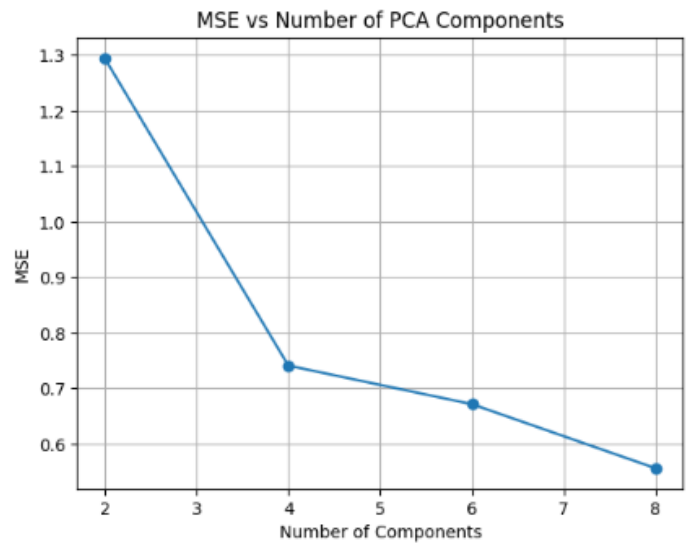
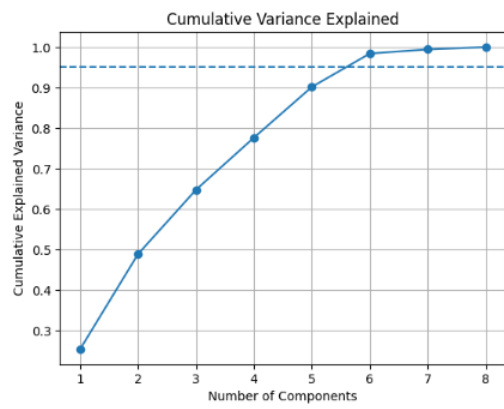
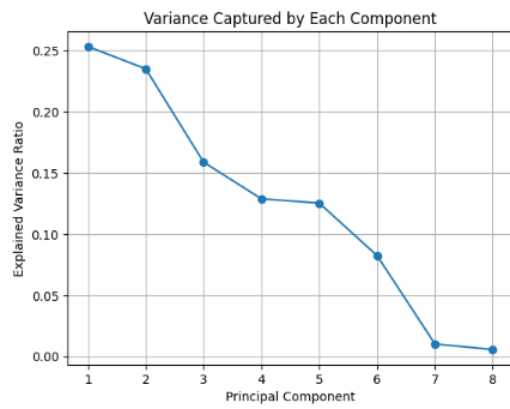
plt.figure()

```

```
plt.plot(range(1, len(cumulative)+1), cumulative, marker='o')  
plt.xlabel("Number of Components")  
plt.ylabel("Cumulative Explained Variance")  
plt.title("Cumulative Variance Explained")  
plt.axhline(y=0.95, linestyle='--') # 95% threshold  
plt.grid()  
plt.show()
```

---- Plot 3: PCA components vs MSE ----

```
plt.figure()  
plt.plot(components, mse_list, marker='o')  
plt.xlabel("Number of Components")  
plt.ylabel("MSE")  
plt.title("MSE vs Number of PCA Components")  
plt.grid()  
plt.show()
```



F 1.